Physics-Informed Neural Networks and Deep Operator Network for beginners

Carlos J. García Cervera[0000−0002−8880−2782],
Mathieu Kessler[0000−0002−0196−5811],
Pablo Pedregal[0000−0001−7814−7895] and
Francisco Periago[0000−0002−7323−1809]

Abstract This chaper is designed to provide graduate students and researchers with an introduction to state-of-the-art, data-driven algorithms for the numerical approximation of a wide array of control and optimization problems governed by both linear and nonlinear partial differential equations (PDEs). Coverage includes a comprehensive exploration of the theory and convergence analysis of Physics-Informed Neural Networks (PINNs) and Deep Operator Network (DeepONet). PINNs are applied to problems where initial and boundary conditions remain fixed, while DeepONet is used to approximate operators that map these conditions to optimal controls or designs.The numerical implementation of these algorithms using Python is considered as well. Python codes for the examples addressed in the course are available in `https://github.com/fperiago/pinn_deeponet_for_beginners`

1 Introduction

During the last decade there has been a deep research effort to develop numerical methods for solving PDEs-based mathematical models by using techniques from Ma-

Carlos J. García Cervera
Department of Mathematics, University of California, Santa Barbara, CA 93106, USA, and BCAM-Basque Center for Applied Mathematics, Bilbao E-48009, Basque Country, Spain; e-mail: `cgarcia@ucsb.edu`

Mathieu Kessler
Department of Applied Mathematics and Statistics, Universidad Politécnica de Cartagena, 30302 Cartagena, Spain; e-mail: `mathieu.kessler@upct.es`

Pablo Pedregal
Departament of Mathematics, Universidad de Castilla-La Mancha, 13071 Ciudad Real, Spain; e-mail: `pablo.pedregal@uclm.es`

Francisco Periago
Department of Applied Mathematics and Statistics, Universidad Politécnica de Cartagena, 30302 Cartagena, Spain; e-mail: `f.periago@upct.es`

chine Learning (ML). The main motivation is not to try to figure out algorithms that outperform (in terms of accuracy) classical methods like finite differences, finite elements or finite volumes in low spatial dimensions, but numerically approximate high-dimensional problems, where the above-mentioned methods get stuck by the well-known phenomenon of the curse of dimensionality, according to which the complexity of the algorithms grows exponentially with the spatial dimension. Kinetic models, computational finance, computational quantum chemistry, game theory, multiscale or multiphysics modelling, and parameterized equations are just a few examples where high dimensionality is present. In this respect, ML-based methods are a promising tool to deal with these high-dimensional situations.

Apart from the issue of high dimensionality, the methods of ML are easy to implement for a large class of mathematical models. Therefore, they may be helpful to:

- test new models very easily,
- incorporate experimental data in your models, and
- have at one's disposal very quick predictions on your models, maybe not so accurate, but enough for your purposes, which can also be used as an initial guess for a more accurate model.

Among the different data-driven methods for solving PDEs-based models (see [1, 2, 3, 13] and the references therein), the focus in these notes is placed on Physics-Informed Neural Networks (PINNs) [25], and on Deep Operator Network (DeepONet) [19]. Whereas the former aims to approximate functions, e.g. the solution of a PDE for given inital/boundary conditions, the latter tackles the problem of approximating operators, e.g. the solution map of a PDE, where the operator maps inital/boundary conditions to solutions of the system. The computer implementation of both PINNs and DeepONet will be addressed as well. Precisely, we will rely on the Python package Deep-XDE [20].

The rest of the document is structured as follows: Section 2 is a brief introduction to Machine Learning, specifically to regression problems in supervised learning. Section 3 is concerned with PINNs. For pedagogical reasons, we will describe the method by using two concrete examples: the Laplace-Poisson equation, and the exact controllability of the wave equation. Although some information will be provided on convergence analysis, the focus will be placed on the practical use of PINNs. In particular, some shortcomings will be highlighted and suggestions on how to overcome those will be described as well. Section 4 is devoted to DeepONet. This algorithm will be described by using the particular example of the operator that maps the initial conditions of the wave equation to its exact control. In both cases, PINNs and DeepONet, the selection of the neural networks that are used as approximators is properly justified.

2 A brief introduction to Machine Learning

Machine Learning encompasses a broad range of problems, typically grouped into categories based on the type of task or data involved. Here, we focus on *supervised learning*, where to goal is to approximate (or learn) an unknown function $f^* : \mathbb{R}^d \to \mathbb{R}^N$

from a labeled dataset

$$S = \left\{ \left(\boldsymbol{x}_i, \boldsymbol{y}_i = f^*(\boldsymbol{x}_i) \right), 1 \leq i \leq n \right\}. \tag{1}$$

In the technical jargon of ML, the data $\boldsymbol{x}_i$ are referred to as *features* and $\boldsymbol{y}_i$ are the *labels*. Depending on whether or not f^* takes continuous values, the following two cases are considered:

- *regression:* f^* takes continuous values, and
- *classification:* f^* takes discrete values.

Long story short, the standard procedure for regression is as follows:

1. Choose a hypothesis space $\mathcal{H}_m$, i.e. an approximation space. *Artificial neural networks* is the model of choice in Machine Learning. By using the *training dataset* (1), an element $f(\boldsymbol{\theta}; \boldsymbol{x}_i) \in \mathcal{H}_m$ is selected. $\boldsymbol{\theta}$ are the m unknown (or trainable) parameters.
2. Choose a loss function. If we are interested in fitting the data, a popular choice is the so-called *Mean Squared Error* (also called *training error* because it is constructed from the training dataset (1))

$$\hat{\mathcal{R}}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \left(f(\boldsymbol{\theta}; \boldsymbol{x}_i) - f^*(\boldsymbol{x}_i) \right)^2, \quad f \in \mathcal{H}_m. \tag{2}$$

   Nonetheless, other types of loss functions may be used as well (see [6] for a recent survey).
3. Choose an optimization algorithm for computing the optimal parameters $\boldsymbol{\theta}^\star$ that minimize the loss function (2).
4. $f(\boldsymbol{\theta}^\star; \cdot)$ is the *prediction model* that is used to predict (approximate) $f^*(x)$ on new (unseen) data x.

The overall objective is to minimize the *generalization error*

$$\mathcal{R}(f) = \mathbb{E}_{\boldsymbol{x} \sim \mathbb{P}} \left(f(\boldsymbol{\theta}; \boldsymbol{x}_i) - f^*(\boldsymbol{x}_i) \right)^2, \quad x_i \in \mathbf{x}, , f \in \mathcal{H}_m, \tag{3}$$

where $\mathbb{P}$ is an unknown probability distribution of the whole dataset to which the training dataset (1) belongs to.

A canonical example of an hypothesis space $\mathcal{H}_m$ is the one composed of the so-called *Multi-Layer Perceptron (MLP).* See Figure 1 for a graphical illustration.

To each input $\boldsymbol{x} = (x_1, \cdots, x_d) \in \mathbb{R}^d$ it associates the output $\boldsymbol{y} = f_m(\boldsymbol{x}) := \boldsymbol{x}^m$ defined by

$$\begin{cases} \text{input layer:} & \boldsymbol{x}^0 = \boldsymbol{x}, \\ \text{hidden layers:} & \boldsymbol{x}^{k+1} = \sigma\left(\omega^{k+1} \boldsymbol{x}^k + b^{k+1} \right) \quad \text{for } k = 0, 1, \cdots, m-2 \\ \text{output layer:} & \boldsymbol{x}^m = \omega^m \boldsymbol{x}^{m-1} + b^m, \end{cases} \tag{4}$$

where the trainable (optimizable) parameters $\boldsymbol{\theta}$ of the network are composed of the matrices of *weights* $\omega^k \in \mathbb{R}^{d_{k+1} \times d_k}$ and the vectors of *biases* $b^k \in \mathbb{R}^{d_k}$. The *depth* of the neural network is m, and for any k, the vector $\boldsymbol{x}^k \in \mathbb{R}^{d_k}$, with d_k the *width* of

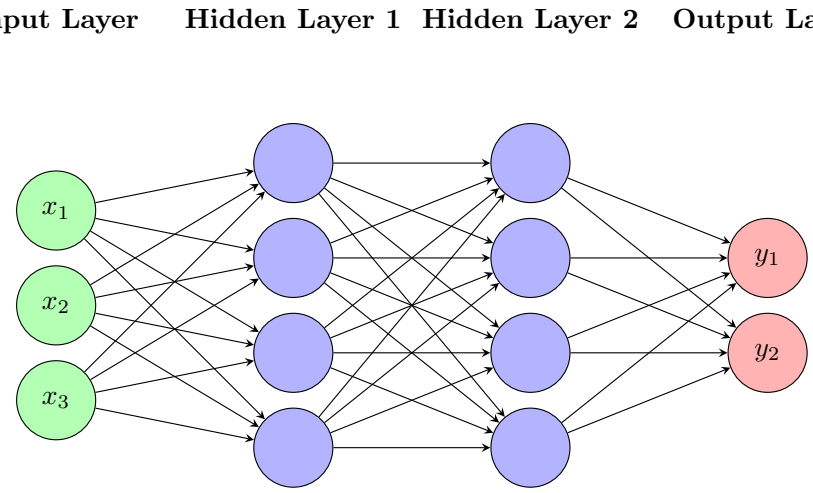


Fig. 1 Illustration of a canonical example of a Multi-Layer Perceptron. It has 3 input channels, 2 hidden layers with 4 neurons each one, and 2 output channels.

the layer k. Finally, σ is a nonlinear function, the so-called *activation function*. The Linear Unit Rectifier ($\mathrm{ReLU}(s) = \max\{s, 0\}$) and the hyperbolic tangent ($\tanh(s)$), $s \in \mathbb{R}$, shown in Figure 2 are typical choices in regression problems. Although ReLU is computationally cheaper than the hyperbolic tangent, for regression problems involving PDEs it is advisable to use the hyperbolic tangent since ReLU is not differentiable at zero and has zero second derivative almost everywhere, which is problematic for PDEs involving higher-order derivatives.

3 Physics-Informed Neural Networks (PINNs)

This section provides a gentle introduction to Physics-Informed Neural Networks (PINNs). For the sake of clarity, we present two illustrative examples. The first one is devoted to learning the solution of the Laplace-Poisson equation in arbitrary spatial dimensions. The description of the PINN algorithm is accompanied by Python code that demonstrates how the network is trained to satisfy both the differential equation and the boundary conditions.

The second example addresses a more advanced application: the null controllability of the wave equation. In this case, the PINN framework is adapted to learn a control function that drives the solution of the wave equation to rest at a prescribed final time. This example highlights the flexibility of PINNs in handling inverse problems and control tasks governed by partial differential equations.

Although these notes primarily focus on the practical implementation of PINNs, they also include remarks on convergence analysis and error estimates, along with several recommendations and a discussion of the main drawbacks of the method.

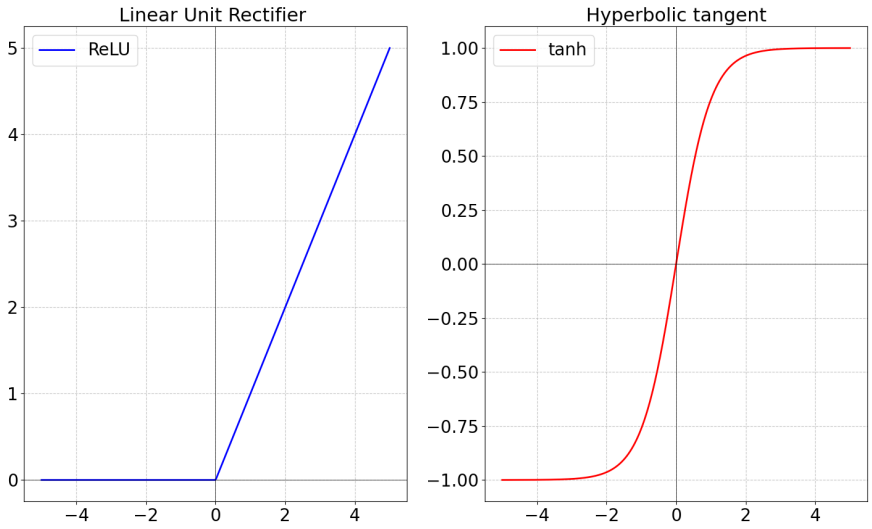


Fig. 2 Linear Unit Rectifier (left) and hyperbolic tangent (right).

3.1 The Laplace-Poisson Equation

Let $d = 1, 2, 3, \cdots$ denote the spatial dimension, and consider the Laplace-Poisson equation

$$\begin{cases} -\Delta u(x) = -2d, & x \in D := (0,1)^d \\ u(y) = \sum_{k=1}^{d} y_k^2, & y \in \partial D \end{cases} \tag{5}$$

This problem has the exact solution

$$u_{\text{exact}}(x) = \sum_{k=1}^{d} x_k^2 \tag{6}$$

Our implementation in `Python` uses the library `DeepXDE` [20]. In addition to `deepxde` we will need `numpy` and `matplotlib`.

In the first step we define the PDE, the geometry and the boundary conditions. Derivatives are computed using Automatic Differentation [4].

```
import deepxde as dde
import numpy as np
import matplotlib.pyplot as plt

d = 3 # spatial dimension

def pde(x, u): # Poisson's equation
    laplacian = sum(dde.grad.hessian(u, x, i=k, j=k) for k in range(d))
    return - laplacian + 2 * d

geom = dde.geometry.Hypercube([0] * d, [1] * d)

# Boundary condition: u(x) = sum x_k^2
def boundary(x, on_boundary):
    return on_boundary

def boundary_value(x):
    return np.sum(np.square(x), axis=1, keepdims=True)

# Boundary condition
bc = dde.icbc.DirichletBC(geom, boundary_value, boundary)
```

Next, we introduce the data, which are composed of the geometry, the partial differential equation, boundary conditions, number of *training* quasi-random points in the interior of the domain, and number of quasi-random points on the boundary. The number of quasi-random *test points* that are used to check accuracy when solving the PDE may be indicated as well. By default, the low-discrepancy Hammersley distribution [7] is used to select those points. It is a good option in low to moderate dimensions (e.g. 2 to 10) because it fills the space more uniformly than random sampling, and it is easy to generate as well. This uniformity reduces the clustering and gaps that are common in purely random samples. In higher dimensions, Sobol or Halton sequences might be more suitable. See Figure 3 for an illustration.

```
data = dde.data.PDE(geom, pde, bc, num_domain=100, num_boundary=40, num_test=100)

train_points = data.train_x  # Training points
test_points = data.test_x    # Test points

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(train_points[:, 0], train_points[:, 1], train_points[:, 2], s=10,
    label='Training Points', alpha=0.6)
ax.scatter(test_points[:, 0], test_points[:, 1], test_points[:, 2], s=10, label='
    Test Points', alpha=0.6, color='red')
plt.legend()
plt.grid(True)
plt.show()
```

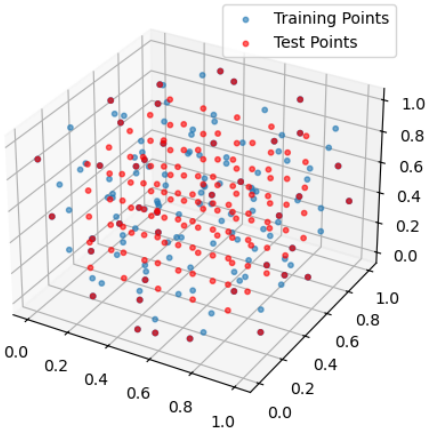


Fig. 3 Training and test points in the domain $D = (0,1)^3$ obtained from the Hammersley distribution.

Next step is to select a network architecture to be used to approximate the solution of the PDE. In this case, we choose a feed-forward neural network where the input layer has dimension d, there are two hidden layers with 50 neurons in each one, and the output is an scalar. As suggested in [29], the hyperbolic tangent is choosen as activation function for all layers. The initialization of the parameters of the network is carried out by following a Glorot uniform distribution. The model can now be built, which accounts for the data and the net.

The loss function is composed of the L^2 residuals for the initial, boundary conditions, and the PDE (in this order). Numerical integration is performed by computing the mean of the loss function evaluated at the training (and test) dataset. More precisely, denoting by $N_{\text{interior}}, N_{\text{boundary}}$ the number of training points in the interior of the domain, and on the boundary, respectively, then

$$\text{loss}(\boldsymbol{\theta}) = \frac{\alpha_1}{N_{\text{interior}}} \sum_{j=1}^{N_{\text{interior}}} |\Delta u(x^j) - 2d|^2 + \frac{\alpha_2}{N_{\text{boundary}}} \sum_{j=1}^{N_{\text{boundary}}} |u(y^j) - \sum_{k=1}^{d} (y_k^j)^2|^2,$$

where x^j and y^j are the training points in the interior and on the boundary, respectively. The weights α_1, α_2 for the PDE and boundary residuals may be selected. The default value is 1, i.e. $\alpha_1 = \alpha_2 = 1$.

The adaptive with moment algorithm (Adam) [15] is the optimizer of choice in deep learning, at least for the type of problems we are addressing here. The suggested learning rate is $lr = 0.001$. After using Adam, it is conveniente to run a quasi-Newton method like L-BFGS in the final stages of training because it is a second-order method that speed up convergence near a local minimim.

```
    net = dde.nn.FNN([d] + [50] * 3 + [1], "tanh", "Glorot uniform")
    model = dde.Model(data, net)
    model.compile("adam", lr=0.001, loss_weights=[1.0, 1.0])
    model.train(iterations=10000)
    model.compile("L-BFGS")
    losshistory, train_state = model.train()
    dde.saveplot(losshistory, train_state, issave=True, isplot=True)
```

Finally, we may compare the PINN and the exact solutions (see Table 1).

```
        # Predict
        X = geom.uniform_points(1000)
        y_pred = model.predict(X)
        print(X.shape)

        # Exact solution
        y_exact = np.sum(X ** 2, axis=1, keepdims=True)

        # Comparison
        error = np.linalg.norm(y_pred - y_exact, 2) / np.linalg.norm(y_exact, 2)
        print("Relative L2 error:", error)
```

Table 1 L^2- relative error $\frac{\|u_{\text{exact}}-u_{\text{PINN}}\|_{L^2(D)}}{\|u_{\text{exact}}\|_{L^2(D)}}$ for different values of the spatial dimension d and number of training points $N = N_{\text{interior}} + N_{\text{boundary}}$

	$N = 100 + 40$	$N = 1000 + 400$	$N = 10000 + 4000$
$d = 2$	9×10^{-4}	1.2×10^{-4}	7.5×10^{-5}
$d = 3$	1.2×10^{-3}	1.4×10^{-4}	1.3×10^{-3}
$d = 5$	2.5×10^{-2}	7.5×10^{-4}	1.9×10^{-4}
$d = 10$	2.2×10^{-1}	4.2×10^{-3}	2.2×10^{-3}
$d = 20$	3.1×10^{-1}	2.6×10^{-2}	2.5×10^{-3}

3.2 Null Controlability of the Wave Equation

Let $\Omega \subset \mathbb{R}^d$ be a smooth bounded domain with boundary $\Gamma = \Gamma_D \cup \Gamma_C$, $\Gamma_D \cap \Gamma_C = \emptyset$, and $T > 0$. The null controllability problem for the wave equation amounts to find a control $v(x,t)$ such that the solution $y(x,t)$ of the system

$$\begin{cases} y_{tt} - \Delta y = 0, & \text{in } Q_T := \Omega \times (0,T) \\ y(x,0) = y^0(x), & \text{in } \Omega \\ y_t(x,0) = y^1(x) & \text{in } \Omega \\ y(x,t) = 0, & \text{on } \Gamma_D \times (0,T) \\ y(x,t) = u(x,t) & \text{on } \Gamma_C \times (0,T) \end{cases} \tag{7}$$

satisfies

$$y(x,T) = y_t(x,T) = 0, \quad x \in \Omega.$$

Following the general procedure describe in Section 2 a surrogate $\hat{y}(x,t;\boldsymbol{\theta})$ of the state variable $y(x,t)$ is constructed as shown schematically in Figure 4 where

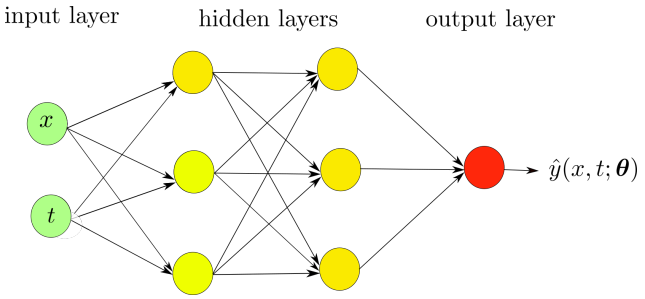


Fig. 4 Illustration of a fully connected deep feedforward neural network with two input channels $\boldsymbol{x} = (x,t)$, two hidden layers (each one with 3 neurons) and an scalar output $\hat{y}(x,t;\boldsymbol{\theta})$. Input data pass through the net by following (16). Figure taken from [11].

$$\begin{cases} \text{input layer:} & \mathcal{N}^0(\boldsymbol{x}) = \boldsymbol{x} = (x,t) \in \mathbb{R}^{d+1} \\ \text{hidden layers:} & \mathcal{N}^\ell(\boldsymbol{x}) = \sigma\left(\omega^\ell \mathcal{N}^{\ell-1}(\boldsymbol{x}) + b^\ell\right) \in \mathbb{R}^{N_\ell}, \quad \ell = 1, \cdots, L-1 \\ \text{output layer:} & \hat{y}(\boldsymbol{x};\boldsymbol{\theta}) = \mathcal{N}^L(\boldsymbol{x}) = \omega^L \mathcal{N}^{L-1}(\boldsymbol{x}) + b^L \in \mathbb{R} \end{cases}$$

- $\mathcal{N}^\ell : \mathbb{R}^{d_{in}} \to \mathbb{R}^{d_{out}}$ is the ℓ layer with N_ℓ neurons,
- $\omega^\ell \in \mathbb{R}^{N_\ell \times N_{\ell-1}}$ and $b^\ell \in \mathbb{R}^{N_\ell}$ are, respectively, the weights and biases so that $\boldsymbol{\theta} = \left\{\omega^\ell, b^\ell\right\}_{1 \le \ell \le L}$ are the parameters of the neural network, and
- σ is an activation function, e.g. $\sigma(s) = \tanh(s)$.

Next step is to select a training dataset $\mathcal{T}$. It is composed of scattered points in the interior domain $\mathcal{T}_{\text{int}} \subset Q_T$ and on the boundaries $\mathcal{T}_{\Gamma_D} \subset \Gamma_D \times (0,T)$, $\mathcal{T}_{t=0} \subset \Omega \times \{0\}$, $\mathcal{T}_{t=T} \subset \Omega \times \{T\}$. Thus, $\mathcal{T} = \mathcal{T}_{\text{int}} \cup \mathcal{T}_{\Gamma_D} \cup \mathcal{T}_{t=0} \cup \mathcal{T}_{t=T}$. The number of selected points in $\mathcal{T}_{\text{int}}$ is denoted by N_{int}. Analogously, N_b is the number of points on the boundary Γ_D, and N_0 and N_T stand for the number of points in $\mathcal{T}_{t=0}$ and $\mathcal{T}_{t=T}$, respectively. The total number of collocation nodes is denoted by N and we write $\mathcal{T}_N$ instead of $\mathcal{T}$ to indicate clearly the number of points N used hereafter. Figure 5 depicts an example of dataset which follows a Sobol distribution [26].

As is usual, the loss function $\mathcal{L}(\boldsymbol{\theta};\mathcal{T})$ is composed of a weighted summation of the L^2 norm of residuals for the equation, boundary, initial and final conditions, i.e.

$$
\begin{aligned}
\mathcal{L}_{\text{int}}\left(\boldsymbol{\theta};\mathcal{T}_{\text{int}}\right) &= \sum_{j=1}^{N_{\text{int}}} w_{j,\text{int}}|\hat{y}_{tt}(\boldsymbol{x}_j;\boldsymbol{\theta}) - \Delta\hat{y}(\boldsymbol{x}_j;\boldsymbol{\theta})|^2,\ \boldsymbol{x}_j \in \mathcal{T}_{\text{int}} \\
\mathcal{L}_{\Gamma_D}\left(\boldsymbol{\theta};\mathcal{T}_{\Gamma_D}\right) &= \sum_{j=1}^{N_b} w_{j,b}|\hat{y}(\boldsymbol{x}_j;\boldsymbol{\theta})|^2, \qquad \boldsymbol{x}_j \in \mathcal{T}_{\Gamma_D} \\
\mathcal{L}^{\text{pos}}_{t=0}\left(\boldsymbol{\theta};\mathcal{T}_{t=0}\right) &= \sum_{j=1}^{N_0} w_{j,0}|\hat{y}(\boldsymbol{x}_j;\boldsymbol{\theta}) - y^0(\boldsymbol{x}_j)|^2, \qquad \boldsymbol{x}_j \in \mathcal{T}_{t=0} \\
\mathcal{L}^{\text{vel}}_{t=0}\left(\boldsymbol{\theta};\mathcal{T}_{t=0}\right) &= \sum_{j=1}^{N_0} w_{j,0}|\hat{y}_t(\boldsymbol{x}_j;\boldsymbol{\theta}) - y^1(\boldsymbol{x}_j)|^2, \qquad \boldsymbol{x}_j \in \mathcal{T}_{t=0} \\
\mathcal{L}^{\text{pos}}_{t=T}\left(\boldsymbol{\theta};\mathcal{T}_{t=T}\right) &= \sum_{j=1}^{N_T} w_{j,T}|\hat{y}(\boldsymbol{x}_j;\boldsymbol{\theta})|^2, \qquad \boldsymbol{x}_j \in \mathcal{T}_{t=T} \\
\mathcal{L}^{\text{vel}}_{t=T}\left(\boldsymbol{\theta};\mathcal{T}_{t=T}\right) &= \sum_{j=1}^{N_T} w_{j,T}|\hat{y}_t(\boldsymbol{x}_j;\boldsymbol{\theta})|^2, \qquad \boldsymbol{x}_j \in \mathcal{T}_{t=T},
\end{aligned}
$$

where $w_{j,\text{int}}$, $w_{j,b}$, $w_{j,0}$ and $w_{j,T}$ are the weights of suitable quadrature rules. Putting all together,

$$
\begin{aligned}
\mathcal{L}\left(\boldsymbol{\theta};\mathcal{T}\right) = &\lambda_1\mathcal{L}_{\text{int}}\left(\boldsymbol{\theta};\mathcal{T}_{\text{int}}\right) \\
&+\lambda_2\mathcal{L}_{\Gamma_D}\left(\boldsymbol{\theta};\mathcal{T}_{\Gamma_D}\right) \\
&+\lambda_3\mathcal{L}^{\text{pos}}_{t=0}\left(\boldsymbol{\theta};\mathcal{T}_{t=0}\right) + \lambda_4\mathcal{L}^{\text{vel}}_{t=0}\left(\boldsymbol{\theta};\mathcal{T}_{t=0}\right) \\
&+\lambda_5\mathcal{L}^{\text{pos}}_{t=T}\left(\boldsymbol{\theta};\mathcal{T}_{t=T}\right) + \lambda_6\mathcal{L}^{\text{vel}}_{t=T}\left(\boldsymbol{\theta};\mathcal{T}_{t=T}\right).
\end{aligned} \tag{8}
$$

Notice that labels equal zero in (8).

The final step in the PINN algorithm amounts to minimizing (8), i.e.,

$$
\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathcal{L}\left(\boldsymbol{\theta};\mathcal{T}\right). \tag{9}
$$

The approximation $\hat{u}(t;\boldsymbol{\theta}^*)$ of the control $u(x,t)$ is then obtained as the restriction of $\hat{y}(x,t;\boldsymbol{\theta}^*)$ to the boundary Γ_C, i.e.

$$
\hat{u}(x,t;\boldsymbol{\theta}^*) = \hat{y}(x,t;\boldsymbol{\theta}^*), \quad x \in \Gamma_C,\ 0 \le t \le T. \tag{10}
$$

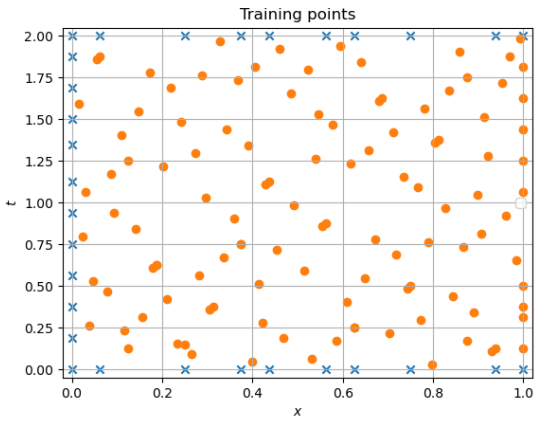


Fig. 5 Illustration of a training dataset (based on Sobol points) in the domain $Q_2 = (0,1)\times(0,2)$. Interior points are marked with circles and boundary points in blue color. $\mathbf{x}_j = (x_j, t_j)$ are the features. Figure taken from [11].

See Figure 6 for an schematic diagram of the proposed algorithm.

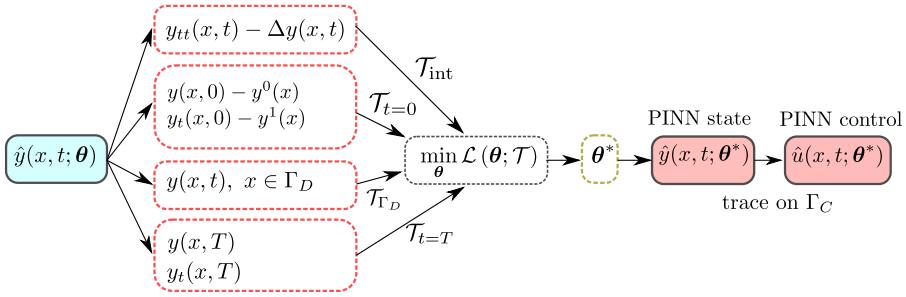


Fig. 6 PINN algorithm for approximating the exact state and control for the wave equation. The neural network $\hat{y}(x,t,\boldsymbol{\theta})$ is required to satisfy, in the least squares sense, the PDE, initial conditions, boundary condition and exact controllability conditions. Then, the residual on training points $\mathcal{L}(\boldsymbol{\theta};\mathcal{T})$ is minimized to get the optimal set of parameters $\boldsymbol{\theta}^*$ of the neural network. This leads to the PINN exact state $\hat{y}(x,t;\boldsymbol{\theta}^*)$. Finally, the PINN exact control $\hat{u}(x,t;\boldsymbol{\theta}^*)$ is obtained as the trace of the PINN state on the boundary control region Γ_C. Figure taken from [11].

3.3 Approximation theory and convergence analysis

A first general question that naturally arises is:

Why is MLP a suitable prediction model for this type of problems?

The answer is found in the celebrated Pinkus' approximation theorem [24, Th. 4.1] that we recall now. Let us consider the hypothesis space of single-layer neural nets

$$\mathcal{H}_m := \left\{ y_m(\boldsymbol{x}) := \sum_{i=1}^{m} a_i \sigma(\boldsymbol{\omega}_i \boldsymbol{x} + b_i) : \boldsymbol{x}, \boldsymbol{\omega}_i \in \mathbb{R}^{d+1}, a_i, b_i \in \mathbb{R} \right\}.$$

Theorem 1 (Pinkus) *Let $f \in C^k(\mathbb{R}^{d+1})$. Assume that the activation function $\sigma \in C^k(\mathbb{R})$ is not a polynomial. Then, for any compact set $K \subset \mathbb{R}^{d+1}$ and any $\varepsilon > 0$ there exists $m \in \mathbb{N}$ and $y_m \in \mathcal{H}_m$ such that*

$$\max_{\boldsymbol{x} \in K} |D^l f(\boldsymbol{x}) - D^l y_m(\boldsymbol{x})| \leq \varepsilon$$

for all multiindex $l \leq k$.

In one space dimension, notice that if σ is a polynomial of degree m, then $\sigma(\omega x + b)$ is a polynomial of total degree at most m. Thus, $\mathcal{H}_m$ is the space of all polynomials of degree m, which is not dense in $C(K)$, K a compact set in $\mathbb{R}$.

To have a first intuition on the converse ([24, Prop. 3.4]), let us assume that $\sigma \in C^\infty(\mathbb{R})$ and consider the space

$$\mathcal{N}(\sigma;\mathbb{R},\mathbb{R}) = \text{span}\{\sigma(wx+b), \quad w,b\in\mathbb{R}\}.$$

A classical result in Calculus (see [8, p. 53])) states that if $\sigma \in C^\infty$ on any open interval and is not a polynomial thereon, then there exists a point $b^\star$ in that interval such that $\sigma^{(k)}(b^\star) \neq 0$ for all $k = 0, 1, 2, \cdots$. For $h \neq 0$,

$$\frac{\sigma\left((w+h)x+b^\star\right)-\sigma\left(wx+b^\star\right)}{h} \in \mathcal{N}(\sigma;\mathbb{R},\mathbb{R})$$

and so

$$\frac{d}{dw}\sigma\left(wx+b^\star\right)|_{w=0} = x\sigma'(b^\star) \in \overline{\mathcal{N}(\sigma;\mathbb{R},\mathbb{R})}.$$

Analogously,

$$\frac{d^k}{dw^k}\sigma\left(wx+b^\star\right)|_{w=0} = x^k\sigma^k(b^\star) \in \overline{\mathcal{N}(\sigma;\mathbb{R},\mathbb{R})}.$$

Since $\sigma^k(b^\star) \neq 0$ for all k, then $\overline{\mathcal{N}(\sigma;\mathbb{R},\mathbb{R})}$ contains all monomials (and also all polynomials). By Weierstrass theorem, $\mathcal{N}(\sigma;\mathbb{R},\mathbb{R})$ is dense in $C(K)$ for any compact set $K \subset \mathbb{R}$.

Pinkus theorem tells us that the space $\mathcal{H}_m$ is dense in $C^k(\mathbb{R}^{d+1})$. Of course, this density result does not imply an efficient approximation scheme. Convergence results for the PINNs schemes described in the two preceding sections may be found in [10, 14, 22, 23].

3.4 Recommendations and Shortcommings

This section briefly reviews on some difficulties encountered by PINNs when solving PDEs-based mathematical models. Some recommendations on its use are also provided. For more details on this passage we refer the reader to [29].

Recommendations.

- **Data normalization.** It is important to ensure that the target output variables vary within a reasonable range. One way to achieve this is through non-dimensionalization of the PDE.
- **Network Architecture.** It is recommended to use Multi-layer Perceptrons (MLPs) with depth and width ranging from 3 to 6, and 128 to 512, respectively. Due to its smoothness, the hyperbolic tangent is the activation function of choice. For initialization of the network parameters, a Glorot uniform distribution is recommended [12].
- **Random weight factorization.** Taking the decomposition for the weights

$$w^{(k,\ell)} = s^{(k,\ell)}v^{(k,\ell)},$$

where $s^{(k,\ell)}$ is a trainable scale factor assigned to each individual neuron, may improve the performance of PINNs [29].

- **Loss balancing.** Assume that when solving a PDE the loss function accounts for the residuals of the initial and boundary conditions, and the PDE itself, i.e.,

$$\mathcal{L}(\boldsymbol{\theta}) = \lambda_{ic}\mathcal{L}_{ic}(\boldsymbol{\theta}) + \lambda_{bc}\mathcal{L}_{bc}(\boldsymbol{\theta}) + \lambda_r\mathcal{L}_r(\boldsymbol{\theta}).$$

  It is advisable to update the weights by following these two steps:

  1. Compute

$$\begin{cases} \hat{\lambda}_{ic} = \dfrac{\|\nabla_\theta \mathcal{L}_{ic}(\theta)\| + \|\nabla_\theta \mathcal{L}_{bc}(\theta)\| + \|\nabla_\theta \mathcal{L}_r(\theta)\|}{\|\nabla_\theta \mathcal{L}_{ic}(\theta)\|} \\ \hat{\lambda}_{bc} = \dfrac{\|\nabla_\theta \mathcal{L}_{ic}(\theta)\| + \|\nabla_\theta \mathcal{L}_{bc}(\theta)\| + \|\nabla_\theta \mathcal{L}_r(\theta)\|}{\|\nabla_\theta \mathcal{L}_{bc}(\theta)\|} \\ \hat{\lambda}_r = \dfrac{\|\nabla_\theta \mathcal{L}_{ic}(\theta)\| + \|\nabla_\theta \mathcal{L}_{bc}(\theta)\| + \|\nabla_\theta \mathcal{L}_r(\theta)\|}{\|\nabla_\theta \mathcal{L}_r(\theta)\|} \end{cases} \tag{11}$$

  2. Update the weights $\lambda = (\lambda_{ic}, \lambda_{bc}, \lambda_r)$ by using a moving average

$$\lambda_{\text{new}} = \alpha\lambda_{\text{old}} + (1-\alpha)\hat{\lambda}_{\text{new}}.$$

- **Optimization algorithm.** By now, the Adaptive with Moment (Adam) algorithm [15] is the most widely used to minimise the loss function. To speed convergence near a local minimum, it is convenient to combine Adam (say, first 20000 iterations) with a quasi-Newton method like L-BFGS (last 5000 iterations).

Shortcommings.

- **Steep gradients.** PINN has difficulty in approximating the solution of PDEs that have steep gradients. A Residual-based Adaptive Refinement (RAR) algorithm has been proposed which adds more residual points in the locations where the PDE residual is large [20].
- **Learning high frecuencies functions.** MLPs are biased towards learning low frecuencies functions. This can be mitigated by using a random Fourier feature embedding [27]. The encoding is

$$\gamma(x) = (\cos(Bx), \sin(Bx)),$$

  where B_{ij} are sampled from Gaussians. Thus, input coordinates are mapped into high feecuency signals before passing through the network.
- **PINNs may violate temporal casuality** when solving time-dependent PDEs. It is advisable to partition the temporal domain into M equal sequential segments. Denoting by $\mathcal{L}_r^i(\theta)$ the PDE residual loss within the i-th segment, the total PDE residual becomes

$$\mathcal{L}_r(\theta) = \sum_{i=1}^{M} \lambda_i \mathcal{L}_r^i(\boldsymbol{\theta}).$$

  See [28] for details.

4 Deep Operator Network (DeepONet)

Given two function spaces X and Y, and an operator (linear or nonlinear)

$$\mathcal{G} : X \to Y$$

the main goal is to learn (approximate) the operator $\mathcal{G}$ from a dataset. DeepONet provides a deep-learning-based algorithm to reach that goal.

As in the preceding section, and for pedagogical reasons, we present the method by considering the operator that maps the initial conditions of the linear wave equation to its exact control.

4.1 Problem Setup

Consider again the control problem

$$\begin{cases} y_{tt} - y_{xx} = 0, & \text{in } (0,1)\times(0,2) \\ y(x,0) = y^0(x), & \text{on } (0,1) \\ y_t(x,0) = y^1(x) & \text{on } (0,1) \\ y(0,t) = 0, & \text{on } (0,2) \\ y(1,t) = u(t) & \text{on } (0,2) \\ y(x,2) = y_t(x,2) = 0, & \text{on } (0,1). \end{cases} \tag{12}$$

The goal is to learn the operator

$$\begin{array}{rl} \mathcal{G} : L^2(0,1)\times H^{-1}(0,1) \to & L^2(0,2) \\ (y^0, y^1) \mapsto & \mathcal{G}(y^0, y^1) := u \end{array} \tag{13}$$

where u is the unique control of minimal L^2-norm of system (12)

It is well-known [9, 17] that the operator $\mathcal{G}$ is well-defined (uniqueness of the control), linear and continuous. Continuity is a consequence of the observability inequality

$$\|u\|_{L^2(0,2)} \le C\left(\|y^0\|_{L^2(0,1)} + \|y^1\|_{H^{-1}(0,1)}\right)$$

Thus, $\mathcal{G}$ is Lipschitz continuous.

The Machine Learning approach for this problem follows these steps:

- **Dataset.** We fix a set of *sensor points* $\{x_1, x_2, \cdots, x_m\} \subset [0,1]$. The information of each selected continuous initial datum (y^0, y^1) is encoded in the vector

$$(y^0(x_1), y^0(x_2), \cdots, y^0(x_m); y^1(x_1), y^1(x_2), \cdots, y^1(x_m)) \equiv y^{\text{initial}}$$

We also take $t \in [0,2]$. Putting all together, our dataset of *features* is

$$\{(y_\ell^{\text{initial}}; t_\ell), \quad 1 \le \ell \le N\} \tag{14}$$

The corresponding *label* is the control at time t_ℓ associated with the initial datum (y^0, y^1), i.e.

$$\{u_\ell = u((y^0, y^1); t_\ell), \quad 1 \le \ell \le N\}, \tag{15}$$

- **Hypothesis space.** We will use the so-called *DeepONet* neural network, which takes the form

$$\mathcal{N}(\boldsymbol{\theta}; (y^{\text{initial}}(x_j); t)) := \sum_{k=1}^{p} \sum_{i=1}^{n} c_i^k \sigma \left(\sum_{j=1}^{m} \xi_{ij}^k y^{\text{initial}}(x_j) + \theta_i^k \right) \cdot \sigma(w_k \cdot t + \eta_k) \tag{16}$$

  where $\boldsymbol{\theta} = (c_i^k, \xi_{ij}^k, \theta_i^k, w_k, \eta_k)$ is the set of (trainable) parameters of the net.
- **Loss function.** As is usual in *regression problems*, we consider the Mean Squared Error between predictions and labels, i.e.

$$\text{MSE}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{\ell=1}^{N} |\mathcal{N}(\boldsymbol{\theta}; (y_\ell^{\text{initial}}; t_\ell)) - u_\ell|^2. \tag{17}$$

4.2 DeepONet's Architecture and Approximation Theory

The operator network $\mathcal{N}$ can be formulated in terms of two shallow ,i.e., one hidden layer, neural networks.The first one is the so-called *branch net*

$$\beta_k(y^{\text{initial}}) = \sum_{i=1}^{n} c_i^k \sigma \left(\sum_{j=1}^{2m} \xi_{ij}^k y^{\text{initial}}(x_j) + \theta_i^k \right), \quad 1 \le k \le p \tag{18}$$

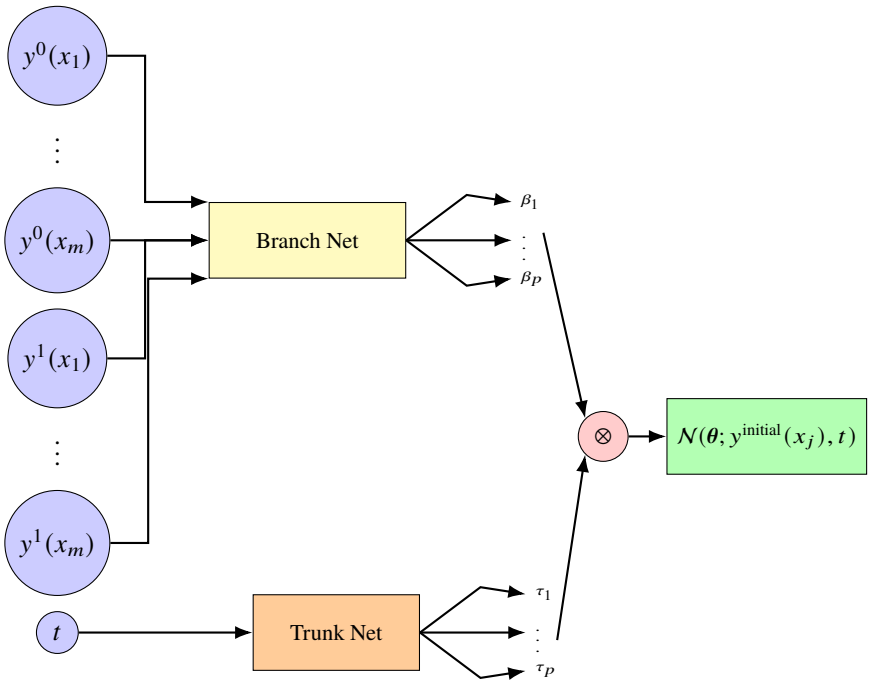
and the second one is the *trunk net*

$$\tau_k(t) = \sigma(w_k \cdot t + \eta_k), \quad 0 \le k \le p \tag{19}$$

so that

$$\mathcal{N}(\boldsymbol{\theta}; (y^{\text{initial}}(x_j); t)) := \underbrace{\sum_{k=1}^{p} \sum_{i=1}^{n} c_i^k \sigma \left(\sum_{j=1}^{m} \xi_{ij}^k y^{\text{initial}}(x_j) + \theta_i^k \right)}_{\beta_k} \cdot \underbrace{\sigma(w_k \cdot t + \eta_k)}_{\tau_k}. \tag{20}$$

Graphically,



From a different perspective, the approximation scheme may be decomposed as

$$\begin{array}{ccc} X & \xrightarrow{\mathcal{G}} & Y \\ \downarrow \mathcal{E} & & \uparrow \mathcal{R} \\ \mathbb{R}^{2m} & \xrightarrow{\mathcal{A}} & \mathbb{R}^p \end{array}$$

where:

- $\mathcal{E} : (y^0, y^1) \mapsto y^{initial}$ is an *encoder*,
- $\mathcal{A} : y^{initial} \mapsto \beta_k(y^{initial})$ is a finite dimensional *approximation operator*, and
- $\mathcal{R} : \beta_k(y^{initial}) \mapsto \sum_{k=1}^{p} \beta_k(y^{\text{initial}})\tau_k(t)$ is a *reconstruction operator.*

Thus, $\mathcal{G} \approx \mathcal{R} \circ \mathcal{A} \circ \mathcal{E}$.

As in the case of PINN, it is natural to ask why this architecture is suitable for operator approximation. We find the answer in the following theorems. For the proofs we refer to [5].

Theorem 2 (Universal Approximation Theorem for Functions) *Suppose that $K \subset \mathbb{R}^d$ is compact, $U \subset C(K)$ is compact, and $\sigma \in C^\infty(\mathbb{R})$ is not a polynomial. Then, for any $\varepsilon > 0$ there exist a positive integer n, real numbers θ_i, $\omega_i \in \mathbb{R}^n$, independent of $f \in U$, and constants $c_i = c_i(f)$ depending on f, such that*

$$|f(x) - \sum_{i=1}^{n} c_i \sigma(\omega_i \cdot x + \theta_i)| < \varepsilon$$

holds for all $x \in K$ and $f \in U$. Moreover, each $c_i(f)$ is a continuous linear functional defined on U.

Crucially, the coefficients $c_i(f)$ are continuous linear functionals. This naturally leads to the question of functional approximation.

Theorem 3 (Universal Approximation Theorem for Functionals) *Suppose that $\sigma \in C^\infty(\mathbb{R})$ is not a polynomial, X is a Banach space, $K \subset X$ is a compact set, V is a compact set in $C(K)$, and $f : V \to \mathbb{R}$ is a continuous functional. Then for any $\varepsilon > 0$, there are a positive integer n, m sensor points $x_1, x_2, \cdots, x_m \in K$, and real constants c_i, θ_i, ξ_{ij}, $1 \le i \le n$, $1 \le j \le m$, such that*

$$|f(y) - \sum_{i=1}^{n} c_i \sigma \left(\sum_{j=1}^{m} \xi_{ij} y(x_j) + \theta_i \right)| < \varepsilon, \quad \text{for all } y \in V.$$

Under suitable continuity and compactness hypotheses, an operator may be approximated from a finite number of functionals. More precisely, if $\mathcal{G}$ is an operator between two function spaces, from the Universal Approximation Theorem for functions we get

$$\mathcal{G}(y)(t) \approx \sum_{k=1}^{n} c_k \left(\mathcal{G}(y)\right) \sigma(\omega_k \cdot t + \theta_k),$$

where $c_k \left(\mathcal{G}(y)\right)$ are continuous functionals. By approximating $c_k \left(\mathcal{G}(y)\right)$ as in the Universal Approximation Theorem for functionals then leads to the DeepONet's architecture (20). Precisely:

Theorem 4 (Universal Approximation Theorem for Operators) *Suppose that $\sigma \in C^\infty(\mathbb{R})$ is not a polynomial, X is a Banach space, $K_1 \subset X$, $K_2 \subset \mathbb{R}^d$ are compact sets, V is a compact set in $C(K_1)$, and $\mathcal{G} : V \to C(K_2)$ is a continuous operator. Then for any $\varepsilon > 0$, there are a positive integers n, p, m sensor points $x_1, x_2, \cdots, x_m \in K_1$, and real constants $c_i^k, \theta_i^k, \xi_{ij}^k, \eta_k$ such that*

$$|\mathcal{G}(y)(t) - \sum_{k=1}^{p} \sum_{i=1}^{n} c_i^k \sigma \left(\sum_{j=1}^{m} \xi_{ij}^k y(x_j) + \theta_i^k \right) \sigma(\omega_k \cdot t + \zeta_k)| < \varepsilon, \quad \forall y \in V, t \in K_2$$

Remark 1 Theorem 4 has been extended to the case of Borel measurable mappings in [16, Theorem 3.1]. Another situation that frequently arises in control theory, particularly when controls are not unique, involves the use of set-valued mappings. This scenario has been recently explored in [10].

4.3 Error Analysis and the Curse of Dimensionality

In the context of Machine Learning, the total error is typically decomposed into approximation, generalization, and optimization errors. Regarding the specific problem of operator approximation, these errors are defined as follows:

- **Approximation error.** This is the distance between the Hypothesis space and the operator $\mathcal{G}$ to be approximated, i.e., if $\mathcal{F}$ is a fixed space of DeepONets, then

$$\mathcal{N}_{\mathcal{F}} = \arg\min_{\mathcal{N}\in\mathcal{F}} \mathcal{L}(\mathcal{N}) := \int_{L^2(D)} \int_U |\mathcal{G}(y)(t) - \mathcal{N}_{\mathcal{F}}(y)(t)|^2 \, dt d\mu(y),$$

  then *approximation error* is

$$\mathcal{E}_{\text{approx}} := \sqrt{\mathcal{L}(\mathcal{N}_{\mathcal{F}})}.$$

- **Generalization (or estimation) error.** This error is due to the fact that we approximate $\mathcal{N}_{\mathcal{F}}$ by using a specific (training) dataset $\mathcal{T}$, and an *empirical loss* $\mathcal{L}_M(N)$, e.g.

$$\mathcal{L}_M(\mathcal{N}) := \frac{|U|}{M} \sum_{j=1}^{M} |\mathcal{G}(y_j)(t_j) - \mathcal{N}(y_j)(t_j)|^2.$$

  Thus, we get

$$\mathcal{N}_{\mathcal{T}} = \arg\min_{\mathcal{N}\in\mathcal{F}} \mathcal{L}_M(\mathcal{N}).$$

  Generalization error is then

$$\mathcal{E}_{\text{gener}} := \left(\mathcal{L}(\mathcal{N}_{\mathcal{T}}) - \mathcal{L}(\mathcal{N}_{\mathcal{F}})\right)^{1/2}.$$

- **Optimization error.** The empirical loss is highly nonlinear and non-convex. Thus, typically we are only able to compute aan approximation of a local minimum $\mathcal{N}_M$ of $\mathcal{L}_M$. *Optimization error* is then

$$\mathcal{E}_{\text{optim}} := \|\mathcal{N}_M - \mathcal{N}_{\mathcal{T}}\|^{1/2},$$

  where $\|\cdot\|$ is an operator norm.

  Putting all together, the total error is

$$\mathcal{E}_{\text{total}} = \mathcal{E}_{\text{approx}} + \mathcal{E}_{\text{gener}} + \mathcal{E}_{\text{optim}}.$$

  We start analysing approximation error. We need the following definitions.

Definition 1 (Data for DeepONet approximation [16]) Assume that $X \hookrightarrow L^2(D)$, and $Y \hookrightarrow L^2(U)$, for some Banach spaces X, Y. The pair $(\mu, \mathcal{G})$ is said to be *data for DeepONet approximation* provided $\mu \in \mathcal{P}_2(X)$ is Borel measurable with finite second moments, there exists a Borel set $A \subset X$, composed of continuous functions, $\mu(A) = 1$, and $\mathcal{G} : X \to Y$ is a Borel measurable mapping with $\|\mathcal{G}\|_{L^2(\mu)} < \infty$.

Definition 2 (Curse of dimensionality [16]) For a given $\varepsilon > 0$, let $\mathcal{N}_\varepsilon$ be a DeepONet such that $\mathcal{E}_{\text{approx}} < \varepsilon$, and

$$\text{size}\,(\mathcal{N}_\varepsilon) \sim \mathcal{O}\left(\varepsilon^{-\vartheta_\varepsilon}\right) \quad \text{for some } \vartheta_\varepsilon \geq 0.$$

Our DeepONet approximation, with underlying measure μ, is said to *incurr a curse of dimensionality* if $\lim_{\varepsilon\to 0}\vartheta_\varepsilon = +\infty$ and *breaks the curse of dimensionality* if $\lim_{\varepsilon\to 0}\vartheta_\varepsilon = \overline{\vartheta} < +\infty$.

In principle, we have bad news since Yarotsky [30] proved that the approximation of a general Lipschitz function to accuracy ε requires a ReLU network of size[1] $\varepsilon^{-m(\varepsilon)/2}$, with $m(\varepsilon)\to\infty$ as $\varepsilon\to 0$, and hence suffers from the curse of dimensionality. In our setting, m is the number of sensors for the enconding operator $u \mapsto \mathcal{E}(y) = (y(x_1), \cdots, y(x_m))$.

However, in some cases, DeepONet approximation may break the curse of dimensionality for approximation error. Some examples are [16, Section 4]:

- holomorphic mappings $[-1,1]^N \to V, y \mapsto \mathcal{G}(y)$ with V an arbitrary Banach space,
- some PDE operators like
  1. Parametric elliptic PDEs: $\mathcal{G} : a \mapsto y$, where y solves
  $$-\nabla\cdot(a\nabla y) = f.$$
  2. Parabolic nonlinear PDEs: $\mathcal{G} : u^0 \mapsto y(T)$, where y solves
  $$\begin{cases} \frac{\partial y}{\partial t} = \Delta y + f(y) \\ y(0) = y^0. \end{cases}$$
  3. Scalar conservation laws: $\mathcal{G} : y^0 \mapsto y(T)$, where y solves
  $$\begin{cases} \partial_t y + \partial_x (f(y)) = 0 \\ y(0) = y^0. \end{cases}$$
- Bounded linear operators $\mathcal{G} : L^2(D) \to L^2(U)$, e.g., the operator (13) defined above.

As for generalization error, under suitable boundedness and Lipschitz continuity assumptions (see [16, Section 5]) one gets

$$\mathcal{E}_{\text{gener}} \le \frac{C}{\sqrt{N}}\left(1 + Cd_\theta \log\left(CB\sqrt{N}\right)\right)^{2\kappa+\frac{1}{2}} \tag{21}$$

where N is the number of sampling functions, d_θ is the number of parameters of the DeepONet, and C, B and κ are suitable constants. Estimate (21) states that generalization error scales (up to a log) like the standard Monte Carlo scaling.

4.4 Implementation issues

In this section we focuss on the DeepONet approximation of the operator (13). The underlying matrix of features corresponds to initial conditions $(y^0(x), y^1(x))$ evaluated

[1] Size of a neural network is understood as the number of non-vanishing parameters of the net.

at a number of sensor points $x_j \in (0,1)$. This is the aforementioned encoding operator $\mathcal{E} : C(0,1) \times C(0,1) \to \mathbb{R}^m \times \mathbb{R}^m$ that maps $(y^0, y^1) \mapsto y^{\text{initial}}$. Such initial conditions may be generated by sampling a Gaussian random field

$$a(x,\omega) = \sum_{i=1}^{\infty} \sqrt{\lambda_i} e_i(x) \xi_i(\omega), \tag{22}$$

i.e. we choose a probability measure $\mu \in \mathcal{P}(C(0,1))$ in Definition 1 whose probability law is given by the Karhunen-Loève expansion (22). $\xi_j \sim \mathcal{N}(0,1)$ are i.i.d. standard Gaussian variables, and (λ_j, φ_j) are the eigenpairs associated with the squared exponential covariance function

$$C(x,x') = \sigma^2 \exp(-\frac{(x-x')^2}{2\ell^2}). \tag{23}$$

We then truncate (22) and sample the Gaussian random variables. Remember that by Mercer's theorem (see, e.g.[21]) $\{\varphi_j\}$ is an ortonormal basis of $L^2(0,1)$.

The vector of labels corresponds to the exact control, which is explicitly given by

$$u(t) = \begin{cases} \frac{1}{2} y^0(1-t) + \frac{1}{2} \int_{1-t}^{1} y^1(s)\, ds & 0 \le t \le 1 \\ -\frac{1}{2} y^0(t-1) + \frac{1}{2} \int_{t-1}^{1} y^1(s)\, ds & 1 \le t \le 2. \end{cases} \tag{24}$$

Concerning the implementation in `depxde`, by [18], a data point is a triplet

$$((y^0, y^1), t, \mathcal{G}(y^0, y^1)(t)).$$

Moreover, a specific input (y^0, y^1) appears in multiple data points with different values of t. Thus, the matrix of features for training X_{train} and its associated vector of labels y_{train} take the form

$$X_{\text{train}} = \begin{bmatrix} y^{\text{initial}}_{1,1} & \cdots & y^{\text{initial}}_{1,2m} & t_1 \\ \cdots & \cdots & \cdots & \cdots \\ y^{\text{initial}}_{1,1} & \cdots & y^{\text{initial}}_{1,2m} & t_\ell \\ \hline \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots \\ \hline y^{\text{initial}}_{N,1} & \cdots & y^{\text{initial}}_{N,2m} & t_1 \\ \cdots & \cdots & \cdots & \cdots \\ y^{\text{initial}}_{N,1} & \cdots & y^{\text{initial}}_{N,2m} & t_\ell \end{bmatrix}, \qquad y_{\text{train}} = \begin{bmatrix} u_1(t_1) \\ \cdots \\ u_1(t_\ell) \\ \hline \cdots \\ \cdots \\ \hline u_N(t_1) \\ \cdots \\ u_N(t_\ell) \end{bmatrix}.$$

The implementation in `deepxde` is:

Fitting the model is then similar to the case of PINNs. Precisely,

```
    X_train = (X_train[:, :-1], X_train[:, -1:])
    X_test = (X_test[:, :-1], X_test[:, -1:])

    data = dde.data.Triple(
    X_train=X_train, y_train=y_train, X_test=X_test, y_test=
        y_test
    )
```

```
        m = X_train_input_dimension  # inferred from
            training data

        dim_t = 1  # input dimension of the trunk

        p = 10  # output dimension of the branch and trunk
            nets

        net = dde.nn.DeepONet(
        [m, 40, p],        # dimensions of the branch net
        [dim_t, 40, p],    # dimensions of the trunk net
        "relu",            # activation function
        "Glorot normal",   # initialization of parameters
        )

        model = dde.Model(data, net)

        model.compile("adam", lr=0.001)

        losshistory, train_state = model.train(iterations=
            ITERATIONS)

        dde.saveplot(losshistory, train_state, issave=True,
            isplot=True)
```

Reproducibility

Python scripts for all the numerical experiments included in these notes can be downloaded from `https://github.com/fperiago/pinn_deeponet_for_beginners`

Acknowledgements This chapter summarizes the lectures given by Francisco Periago at the XXI Jacques-Louis Lions Hispano-French School on Numerical Simulation in Physics and Engineering held at Universidad de Castilla-La Mancha (Spain) on July 7-11, 2025. FP acknowledges the invitation by SEMA (Spanish Society of Applied Mathematics) and SMAI (French Society for Applied and Industrial Mathematics). He is also indebted to the organizers for their kindness and hospitality.

Competing Interests C. J. García Cervera acknowledges support from AFOSR (MURI FA9550-18-1-0095).
P. Pedregal has been supported by grants PID2023-151823NB-I00, and SBPLY/23/180225/000023.

M. Kessler and F. Periago have been supported by grant PID2022-141957OA-C22 funded by MCIN/AEI/10.13039/501100011033, by "ERDF A way of making Europe", and the Autonomous Community of the Región of Murcia, Spain, through the programme for the development of scientific and technical research by competitive groups (21996/PI/22), included in the Regional Program for the Promotion of Scientific and Technical Research of Fundación Séneca – Agencia de Ciencia y Tecnología de la Región de Murcia.
The authors have no conflicts of interest to declare that are relevant to the content of this chapter.

References

1. Beck, C., Becker, S., Grohs, P., Jaafari, N., Jentzen, A.: Solving the Kolmogorov PDE by means of deep learning. SIAM J. Sci. Comput. **88**(3) (2021)
2. Beck, C., Martin, H., Jentzen, A., Benno, K.: An overview on deep learning-based approximation methods for partial differential equations. DCDS-B **28**(6), 3697–3746 (2023)
3. Blechschmidt, J., Ernst, O.G.: Three ways to solve partial differential equations with neural networs - A review. GAMM-Mitt **44**(2), 1–29 (2021)
4. Bydin, A.G., Pearlmutter, B.A., Radul, A.A., Siskind, J.M.: Automatic differentation in machine learning: a survey. J. Mach. Learn. Res. **18**, 1–43. (2018)
5. Chen, T., Chen, H.: Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. IEEE Transactions on Neural Networks **6**(4), 911–917 (1995)
6. Ciampiconi, L., Elwood, A., Leonardi, M., Mohamed, A., Rozza, A.: A survey and taxonomy of loss functions in machine learning. https://arxiv.org/abs/2301.05579 (2024)
7. Dick, J., Pillichshammer, F.: Digital Nets and Sequences: Discrepancy Theory and Quasi-Monte Carlo. Cambridge University Press (2010)
8. Donoghue, W.F.: Distributions and Series Transforms. Academic Press, New York (1969)
9. Fernández-Cara, E., Zuazua, E.: On the null controllability of the one-dimensional heat equation with bv coefficients. Comput. Appl. Math. **21**(1), 167–190 (2002)
10. García-Cervera, C.J., Kessler, M., Pedregal, P., Periago, F.: Universal approximation of set-valued maps and application to control. Sumitted. (2025)
11. García Cervera, C.J., Kessler, M., Periago, F.: Control of partial differential equations via physics-informed neural networks. J. Optim. Theory Appl. **196**(2), 391–414 (2023)
12. Glorot, X., Bengio, Y.: Understanding the dificulty of training deep feedforward neural networks. Proceedings of the thirteenth international conference on artificial intelligence and statistics pp. 249–256 (2010)
13. Han, J., Jentzen, A., Weinan, E.: Solving high-dimensional partial differential equations using deep learning. PANS **115**(34), 8505–8510 (2018)
14. Jiao, Y., Lai, Y., Li, D., Lu, X., Wang, F., Null, Y.W., Yang, J.Z.: A rate of convergence of physics informed neural networks for the linear second order elliptic pdes. Commun. Comput. Phys. **31**(4), 1272–1295 (2022)
15. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. 3rd International Conference on Learning Representations (2015)
16. Lanthaler, S., Mishra, S., Karniadakis, G.E.: Error estimates for DeepONets: a deep learning framework in infinite dimensions. Trans. Math. Appl. **6**(1), 1–144 (2022)
17. Lions, J.L.: Controllabilité exacte, perturbations et stabilization de systémes distribués, vol. I. Masson (1988)
18. Lu, L., Jin, P., Pang, G., Zhang, Z., Karniadakis, G.: Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. Nature Machine Intelligence **3**(3), 218–229 (2021)
19. Lu, L., Jin, P., Pang, G., Zhang, Z., Karniadakis, G.E.: Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. Nature Machine Intelligence **3**(3), 218–229 (2021)

20. Lu, L., Meng, X., Mao, Z., Karniadakis, G.E.: DeepXDE: A deep learning library for solving differential equations. SIAM Review **63**(1), 208–228 (2021)
21. Martínez-Frutos, J., Periago, F.: Optimal control of PDEs under uncertainty. An introduction with application to optimal shape design of structures. SpringerBriefs in Mathematics. BCAM SpringerBriefs. Springer (2018)
22. Mishra, S., Molinaro, R.: Estimates on generalization error of physics-informed neural networks for approximating a class of inverse problems for PDEs. IMA J. Numer. Anal. **42**
23. Mishra, S., Molinaro, R.: Estimates on generalization error of physics-informed neural networks for approximating PDEs. IMA J. Numer. Anal. **43**(1), 1–43 (2023)
24. Pinkus, A.: Approximation theory of the MLP model in neural networks. Acta numer. **8**, 143–195 (1999)
25. Raisi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J. Comput. Phys. **378**, 686–707 (2019)
26. Sobol', I.M.: On the distribution of points in a cube and the approximate evaluation of integrals. Zhurnal Vychislitel'noi Matematiki i Matematicheskoi Fiziki **7**(4), 784–802 (1967)
27. Tancik, M., et al.: Fourier features let networks learn high frequency functions in low dimensional domains. 34th Conference on Neural Information Processing Systems (2020)
28. Wang, S., Sankaran, S., Perdikaris, P.: Respecting causality for training physics-informed neural networks. Comput. Methods Appl. Mech. Engrg. **421**, 116813, 17 pp. (2024)
29. Wang, S., Sankaran, S., Wang, H., Perdikaris, P.: An expert's guide to training physics-informed neural networks. https://arxiv.org/abs/2308.08468 (2023)
30. Yarotsky, D.: Optimal approximation of continuous functions by very deep relu networks. Conference on Learning Theory. PMLR pp. 639–649 (2018)